The Spread -Average of H-L trading rule and the rule that supersedes it…

This spread – [H-L][n]/n conditional rule is *exactly* the kind of "institutional execution filter" we want baked into the pipeline, because it prevents the model from "making money on paper" while getting murdered by **liquidity + microstructure** in real fills.

Here's how we formalize what I have described in the message into a **clean, model-friendly rule family**, plus how to make it **learnable/dynamic** without hardcoding thousands of brittle if/then statements. Remember this is one rule of thousands I need to work out and feed what is constructed below into AntiGrav and then work with it to get the execution decision logic correct so the model learns correctly. These steps are critical for accurate and working model training…

---

1) Formalize The Rule as an Execution Gating Primitive

Definitions (per bar *t*)

- **Spread(t)** = Ask(t) − Bid(t) *(for the option I're trading; per leg if multi-leg)*
- **AvgRange(n, t)** = $(1/n) \cdot \Sigma_{i=0..n-1}$ (High(t−i) − Low(t−i))
  (for SPY underlying on 1m bars, or optionally for option mid-price range)

Base Gate

- **If** Spread(t) ≥ AvgRange(n, t) → **BLOCK** entry/exit (no trade)
- **If** Spread(t) < AvgRange(n, t) → **ALLOW** entry/exit

This is a very good "liquidity must be small relative to realized movement" gate.

But we can make it more robust and learnable.

---

2) Improve It: Convert to a Dimensionless Ratio (Cleaner Learning)

Define a **microstructure friction ratio**:

F(t) = Spread(t) / AvgRange(n, t)

Then the rule becomes:

- **BLOCK** if F(t) ≥ 1
- **ALLOW** if F(t) < 1

Now the model doesn't have to learn two scales — it learns one normalized number.

---

3) Add the Learnable Coefficient I Mentioned

I described a coefficient that changes with volatility/regime/news/etc. Perfect.

Make the threshold dynamic:

ALLOW if F(t) < θ(t)

Where θ(t) is a **learned gating threshold** produced by:

- volatility regime (ATR, Bollinger bandwidth, VIX proxy / IV rank),
- liquidity regime (volume ratio, spread percentile),
- event regime (FOMC window, CPI window, open/close session),
- "coming out of consolidation" signals (bandwidth compression → expansion).

We can implement θ(t) as:

- a **simple parametric function** (fast and stable), or
- a **small neural head** (if I want it learned end-to-end).

Example θ(t) (parametric, robust)

Let:

- BW(t) = Bollinger bandwidth (normalized)
- ATRn(t) = ATR / price
- VolRatio(t) = volume / SMA(volume)
- Session(t) = categorical (open/mid/close)
- Event(t) = binary flag (macro window)

Then:
$\theta(t) = \text{clamp}(\theta_0 + a \cdot z(\text{ATRn}) + b \cdot z(\text{VolRatio}) - c \cdot z(\text{BW}) - d \cdot \text{Event}(t), \theta_{min}, \theta_{max})$

Interpretation:

- Higher volatility / more volume → tolerate slightly more spread (θ increases)
- Tight bandwidth + low volume or event risk → be stricter (θ decreases)

This captures exactly what I mean by "learned and dynamically adjusted."

---

4) The "Override" Clause: Hard Exit on Gap Risk

It said: even if spread gate blocks exits, if there's high probability of a gap against the position, exit anyway to avoid catastrophe.

That is correct. This should be a **separate "risk override" gate** that supersedes execution gating.

Define:

Gap Risk Score G(t) ∈ [0,1]

A classifier (or rule-based proxy) that estimates probability of an adverse jump.

Common inputs:

- event flags (FOMC/CPI/earnings/major macro),
- volatility expansion (ATR spike, bandwidth expansion),
- orderflow proxy (volume spike + directional momentum),
- "late-day risk" (near close gaps),
- implied vol steepening / skew changes (if I have it).

Override Rule

If **ExitSignal(t) = true** and **G(t) ≥ G_crit**, then:

- **EXIT IMMEDIATELY at market** (or marketable limit)
  regardless of spread gate.

That's how institutions behave: *execution cost is secondary to gap risk.*

---

5) How This Reduces "Thousands of If/Then Statements"

Instead of encoding thousands of rules, we define a **small number of reusable primitives**:

Core Primitives (Reusable)

1. **Signal Generator(s)**
   (trend/momentum/mean reversion/breakout signals)
2. **Execution Gate**
   (spread vs realized range friction)
3. **Risk Override Gate**
   (gap risk / tail risk / liquidation triggers)

4. **Sizing Gate (FIS)**
   (confidence → position size)
5. **Portfolio Constraint Gate**
   (Greeks/caps/drawdown)

Almost every "if/then" described can be expressed as:

TradeAction = Signal ∧ ExecutionGate ∧ PortfolioGate
unless RiskOverride triggers, then force-exit

This gets I **90%** of institutional behavior with **dozens** of primitives, not thousands of brittle rules.

---

6) How to Feed This into Model Creation (Practical)

Add these **columns/features** into training rows (per bar):

Execution Microstructure Features

- spread_abs = ask−bid
- spread_pct = spread / mid
- avg_range_n = mean(high−low over n)
- friction_ratio F = spread_abs / avg_range_n
- spread_percentile_1d (rolling)
- volume_ratio

Gates (as labels or auxiliary heads)

- exec_allow = $1\{F < \theta(t)\}$
- gap_risk_score (0–1)
- risk_override = $1\{\text{gap\_risk\_score} \geq G_{crit}\}$

Training Targets

- **policy head** predicts: enter/hold/exit (and which structure)
- **aux head** predicts: exec_allow probability
- **aux head** predicts: gap_risk probability

This makes the model "understand" *why* it can't enter/exit and when it must override.

---

7) Clean Rule Spec (Ready to Drop into The Rule Library)

Rule: Execution Friction Gate (EFG-01)

- Compute AvgRange(n) from SPY 1m bars
- Compute option Spread = Ask − Bid
- Compute F = Spread / AvgRange(n)
- Allow entry/exit only if $F < \theta(t)$
- $\theta(t)$ is dynamic, determined by volatility, volume, bandwidth, session, event flags

Rule: Tail Gap Override (TGO-01)

- If ExitSignal is true and GapRiskScore ≥ Gcrit
- Force exit with market order regardless of EFG-01

---

next we need to define **the canonical rule primitives list** (maybe 30–60 primitives total)

- Provide **a template "Rule DSL"** (YAML/JSON style) so AGrav can encode rules cleanly
- Map each primitive to a **feature column + gating output** that the model can learn

And we'll build it so we can keep expanding without creating an unmaintainable if/else jungle.